プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

- テーマ：ボールクラスを作る
- 例題：オブジェクト・クラスの作成
- 課題：ボールクラスを作る

オブジェクト

- 教科書第10章
  オブジェクトは関連しあう変数と関数の集合体
  - オブジェクトはデータと命令を結び付けたもの
  - 関連しあうデータとそれに働きかける命令や振る舞いを一つにまとめる
  - 呼び方
    - 変数　→　フィールド
    - 関数　→　メソッド

クラス

- オブジェクトを作る前にクラスの定義が必要
- クラスを新規タブを使って追加する
- クラスを作る4ステップ

1. (クラスの)ブロックを作成
2. フィールドを追加
3. コンストラクタの生成とフィールドへの代入
4. メソッドの追加

クラスのブロックを作成

- 書き方

```
class クラス名 {

}
```

- classは小文字表記
- クラス名の頭文字は大文字
- 2単語以上のクラス名は
  単語の頭文字を大文字にしてつなげる
- クラス名の後ろに{}(ブロック)

- 例）JitterBugクラス

```
class JitterBug {

}
```

フィールドの追加

- 変数と同じように宣言

```
class クラス名 {
  型 フィールド名;
  型 フィールド名 = 初期値;
}
```

コンストラクタで値を
代入するフィールドは宣言だけ

そうでないフィールドは
初期値を代入

- このときにコンストラクタで値を代入する
  フィールドを決める
  - そうでないフィールドは宣言と同時に初期値を設定

フィールドの追加

- 例）JitterBugクラス

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;
}
```

x, y, diameterは
コンストラクタで値を
代入するので宣言だけ

speedは
コンストラクタで値を代入しないので
初期値を代入

コンストラクタの追加

- クラス名と同じ名前の
  型をつけない特別なメソッド
- コンストラクタの役割
  - オブジェクトが作られるときにフィールドの初期値を代入する
  - コンストラクタの中のコードは、オブジェクト作成時1回だけ実行

コンストラクタの追加

- コンストラクタの書き方

```
class クラス名 {
  型 フィールド名;
  型 フィールド名 = 初期値;

   クラス名(){


   }


}
```

- 型を付けない
- コンストラクタの名前はクラス名と同じ

※この後まだ続きます

フィールドへの代入

- 初期値を設定しなかったフィールドは、コンストラクタの中で初期値を設定する

```
class クラス名 {
  型 フィールド名;
  型 フィールド名 = 初期値;

  クラス名(型 パラメータ名){
    フィールド名 = パラメータ名;
  }

}
```

パラメータの型は
フィールドの型は同じにする

パラメータが複数の場合は
コンマ(,)で区切り
型 パラメータ名を続けて書く

フィールドに
パラメータを代入する

フィールドへの代入

- 例）教科書 P137 JitterBugクラス

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug(float tempX, float tempY, int tempDiameter){
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }
}
```

speedは
コンストラクタで値を代入しないので
初期値を代入

x, y, diameterは
パラメータ tempX, tempY, tempDiameterを
代入する

メソッドの追加

- メソッドの定義

```
class クラス名 {
  型 フィールド名;
  型 フィールド名 = 初期値;

  クラス名(型 パラメータ名){
    フィールド名 = パラメータ名;
  }

  型 メソッド名(型 パラメータ名){

  }

}
```

関数の定義と同じ書き方

メソッドが複数の場合
このブロックを複数書く

メソッドの追加

- 例）JitterBugクラス

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug(float tempX, float tempY, int tempDiameter){
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){
    x += random(-speed, speed);
    y *= random(-speed, speed);
  }

  void display(){
    ellipse(x, y, diameter, diameter);
  }

}
```

メソッド move()の定義

メソッド display()の定義

オブジェクトを作る

- プログラムの中で

  オブジェクトを作る2ステップ

1. オブジェクト変数を宣言
2. newを使ってオブジェクトを生成(初期化)

オブジェクト変数の宣言

- 変数の宣言と基本的には同じ

```
クラス名　オブジェクト変数名;
```

- 変数の宣言　型 変数名;
  - クラス名が型にあたるところ

- 例）JitterBugクラス

```
JitterBug  bug;
```

- クラス名(型)がJitterBug
- オブジェクト変数名がbug

newを使ってオブジェクト生成

- 書き方

```
オブジェクト変数名 = new コンストラクタ(パラメータ);
```

- コンストラクタによっては
  複数パラメータを設定する場合有

- 例）JitterBugクラス

```
void setup(){
  size(480, 120);
  bug = new JitterBug(width/2, height/2, 20);
}
```

- setup()関数内で
  1度だけ実行する

- コンストラクタに3つのパラメータを設定して
  3つのフィールドを初期化している

フィールドへのアクセス

- フィールド

オブジェクト変数名.フィールド名

ドット(.)

- 例）JitterBugクラスのオブジェクトbug

```
bug.x
```

bugのフィールド x

```
bug.diameter
```

bugのフィールド diameter

メソッドへのアクセス

- メソッド

オブジェクト変数名.メソッド名(パラメータ)

ドット(.)

- 例）JitterBugクラスのオブジェクトbug

```
bug.move()
```

bugのメソッド move()

```
bug.display()
```

bugのメソッド display()

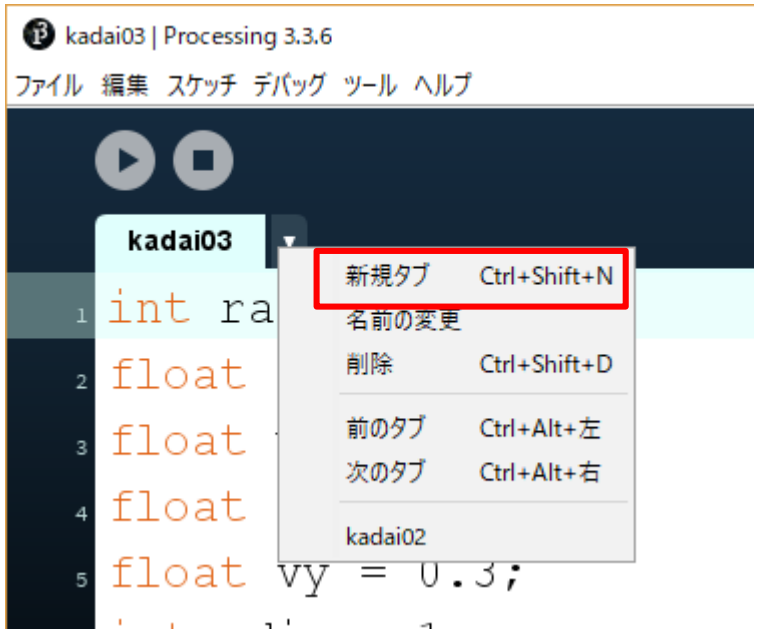
課題Kadai_03 Ballクラスを作る

- 前回課題Kadai_02をもとに、ボールに関連する値と関数をBallクラスとして定義し直して、同じ動作をするプログラムを作る

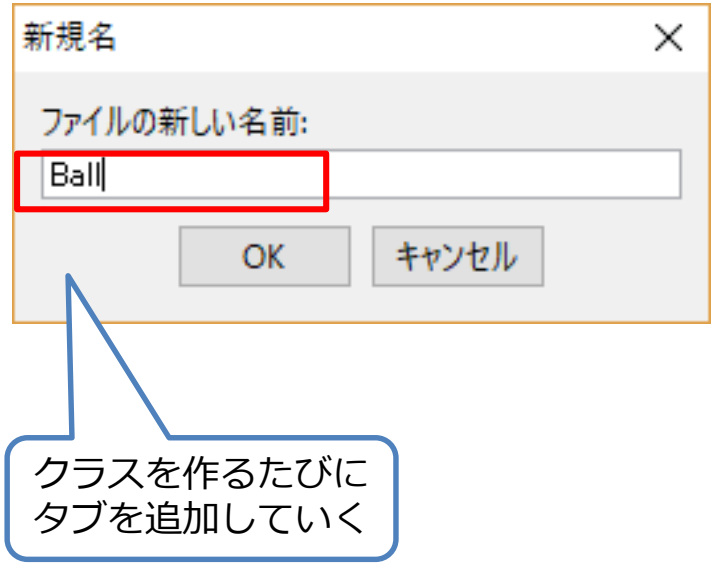
- クラスを作る場合は新規タブを使ってください

新規タブ追加

- 新規タブを選択



- クラス名を入力



Kadai_03

```
//メインタブ
                    //Ball型オブジェクト変数ballの宣言

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);
  ball = new        (110, 60, 0.5, 0.3); //コンストラクタを使ってオブジェクトの生成
}

void draw() {
  background(0);
                                          //ボールの座標を移動する
  if (                  ||                ) { //右壁か左壁に衝突した場合
                                          //ボールのx進行方向を反転する
  }
  if (                  ||                ) { //下壁か上壁に衝突する場合
                                          //ボールのy進行方向を反転する
  }
                                          //ボールを描く
}
```

x座標は110、y座標は60、
x方向の速度は0.5、
y方向の速度は0.3に初期化する

Kadai_03

```
class Ball {
```

フィールドの追加

半径(初期値40)、
x座標、y座標
x方向の速度、y方向の速度
xの進行方向(初期値1)、
yの進行方向(初期値1)

コンストラクタの追加と
フィールドへの代入

コンストラクタで設定するフィールド
x座標、y座標、x方向の速度、y方向の速度

メソッドの追加①
ボールが移動する

メソッドの追加②
x方向に反転する

メソッドの追加③
y方向に反転する

メソッドの追加④
ボールを描く

メソッドの追加⑤
右壁に衝突したかどうか

Kadai_03

メソッドの追加⑥
左壁に衝突したかどうか

メソッドの追加⑦
下壁に衝突したかどうか

メソッドの追加⑧
上壁に衝突したかどうか

}